



**Systems and Software
Verification Laboratory**

MANCHESTER
1824

The University of Manchester

Software Model Checking Tools

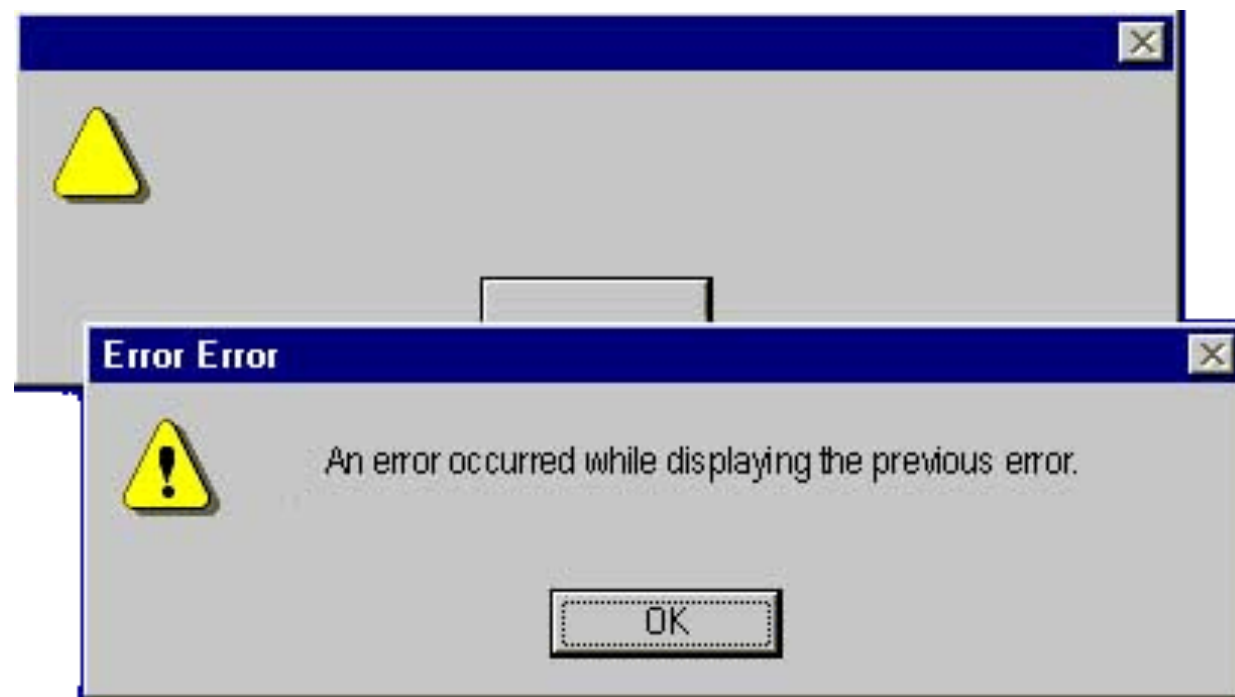
Lucas Cordeiro
Department of Computer Science
lucas.cordeiro@manchester.ac.uk

Software Model Checking Tools

- Lucas Cordeiro (Formal Methods Group)
 - lucas.cordeiro@manchester.ac.uk
 - Office: 2.28
 - Office hours: 15-16 Tuesday, 14-15 Wednesday
- References:
 - Cordeiro et al.: *SMT-Based Bounded Model Checking for Embedded ANSI-C Software*. *IEEE Trans. Software Eng.* 38(4): pp. 957-974, 2012
 - Cordeiro et al.: *JBMC: A Bounded Model Checking Tool for Verifying Java Bytecode*. *CAV*, pp. 183-190, 2018

Intended learning outcomes

- **Introduce** the software verifiers **ESBMC** and **JBMC**
- **Understand** how software verifiers **check safety properties** in C and Java programs
- **Explain** the main **verification strategies** and **property checks** in ESBMC and JBMC
- **Apply** ESBMC and JBMC to **test and verify** C and Java programs



ESBMC v6.0

Mikhail R. Gadelha
Felipe R. Monteiro
Lucas C. Cordeiro
Denis A. Nicole

25th Intl. Conference on Tools and Algorithms for the Construction and Analysis of Systems

8th Intl. Competition on Software Verification

ESBMC v6.0

Verifying C Programs Using k -Induction and Invariant Inference
(Competition Contribution)

Mikhail R. Gadelha, **Felipe R. Monteiro**, Lucas C. Cordeiro, and Denis A. Nicole



UNIVERSITY OF
Southampton



MANCHESTER
1824

ESBMC

Gadelha *et al.*, ASE'18

- SMT-based bounded model checker of single- and multi-threaded C/C++ programs
turned 10 years old in 2018 🎉
- Combines BMC, k -induction and abstract interpretation:
path towards correctness proof
bug hunting
- Exploits SMT solvers and their background theories
optimized encodings for pointers, bit operations, unions,
arithmetic over- and underflow, and floating-points

ESBMC

Gadelha *et al.*, ASE'18

- SMT-based bounded model checker of single- and multi-threaded C/C++ programs

arithmetic under- and overflow

pointer safety

array bounds

division by zero

memory leaks

atomicity and order violations

deadlock

data race

user-specified assertions

built-in properties

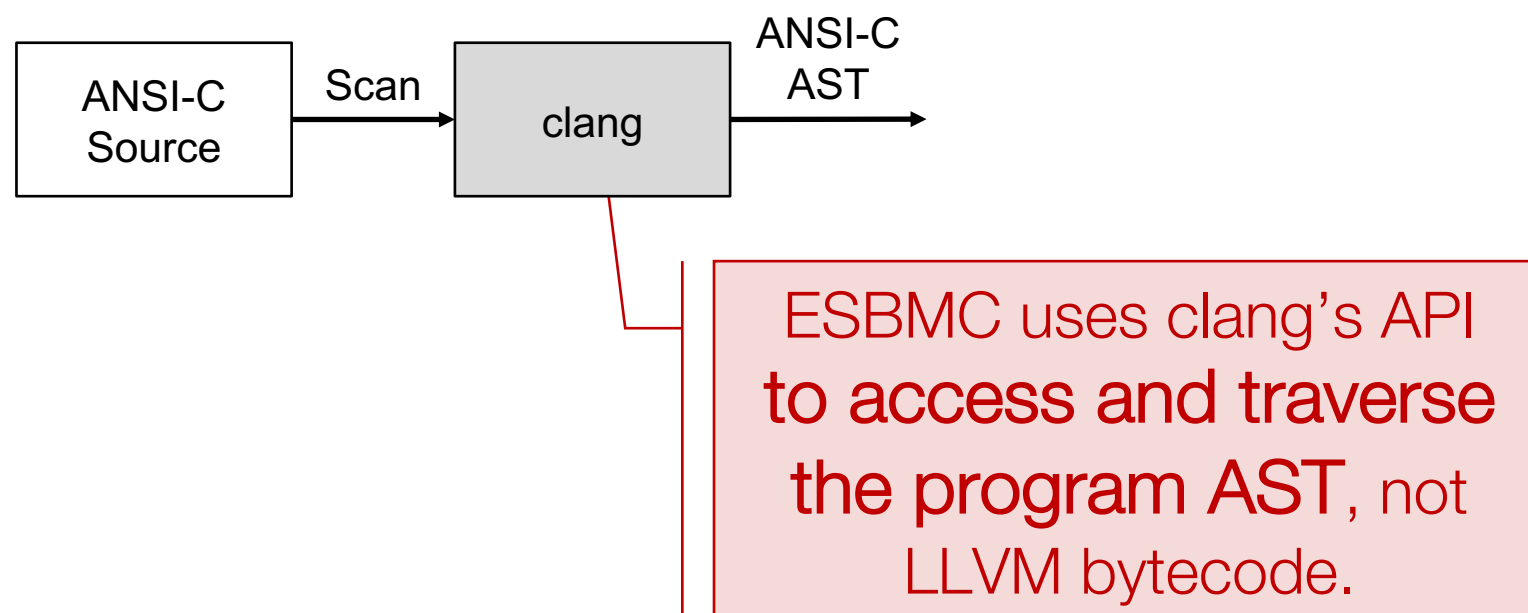
ESBMC Architecture

- ESBMC uses a k -induction proof rule to verify and falsify properties over C programs

ANSI-C
Source

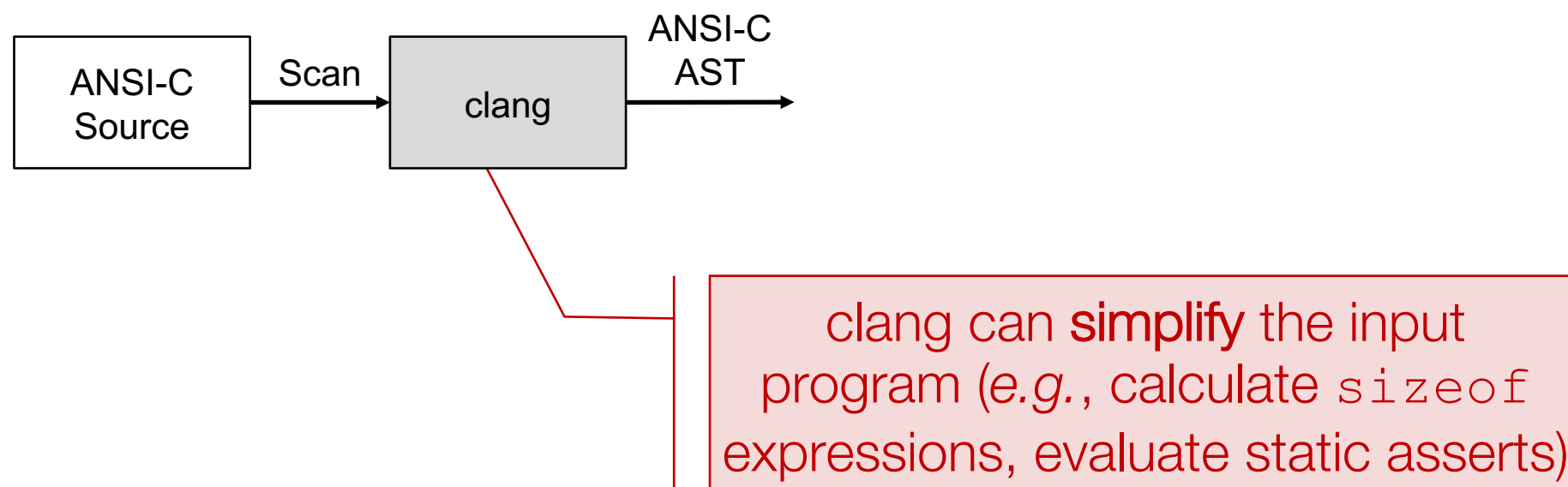
ESBMC Architecture

- ESBMC uses a k -induction proof rule to verify and falsify properties over C programs



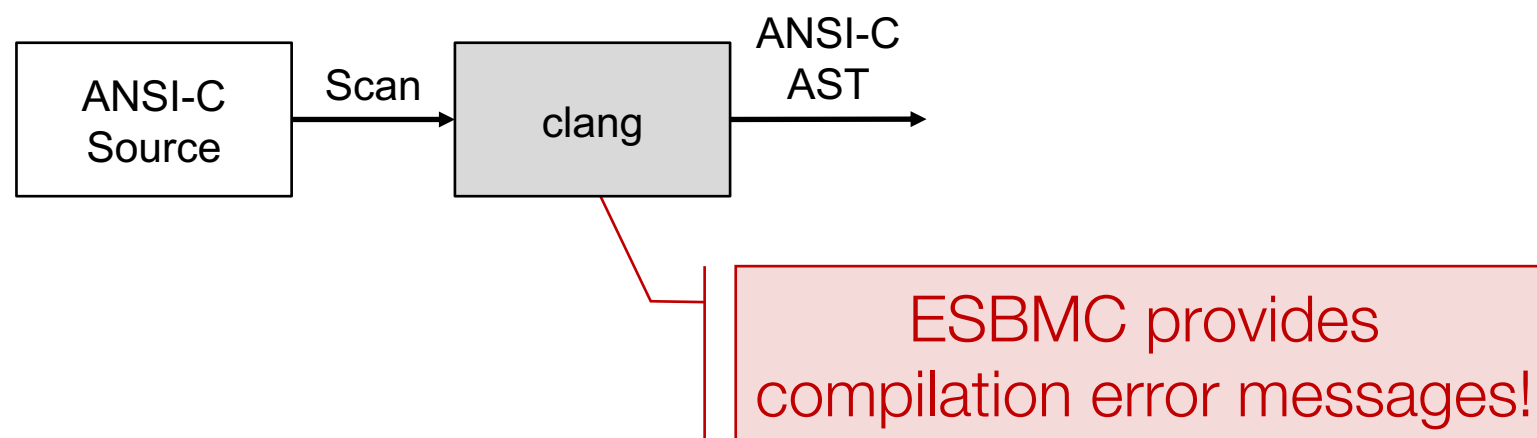
ESBMC Architecture

- ESBMC uses a k -induction proof rule to verify and falsify properties over C programs



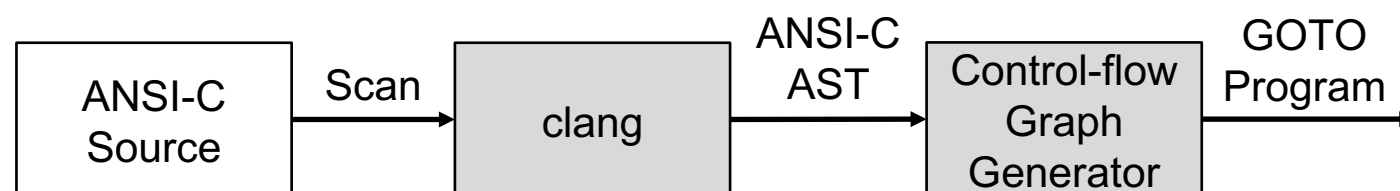
ESBMC Architecture

- ESBMC uses a k -induction proof rule to verify and falsify properties over C programs



ESBMC Architecture

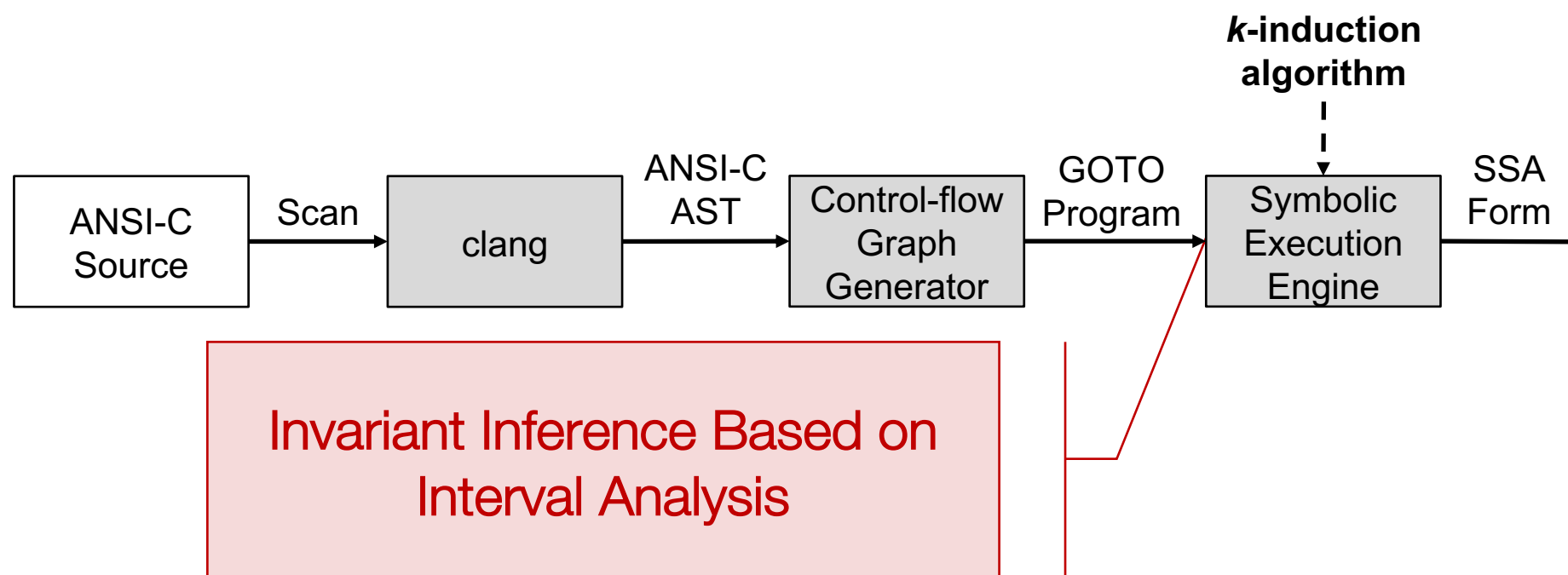
- The CFG generator takes the program AST and transforms it into an equivalent GOTO program
 - only of assignments, conditional and unconditional branches, assumes, and assertions.



ESBMC Architecture

- ESBMC perform a static analysis prior to loop unwinding and (over-)estimate the range that a variable can assume

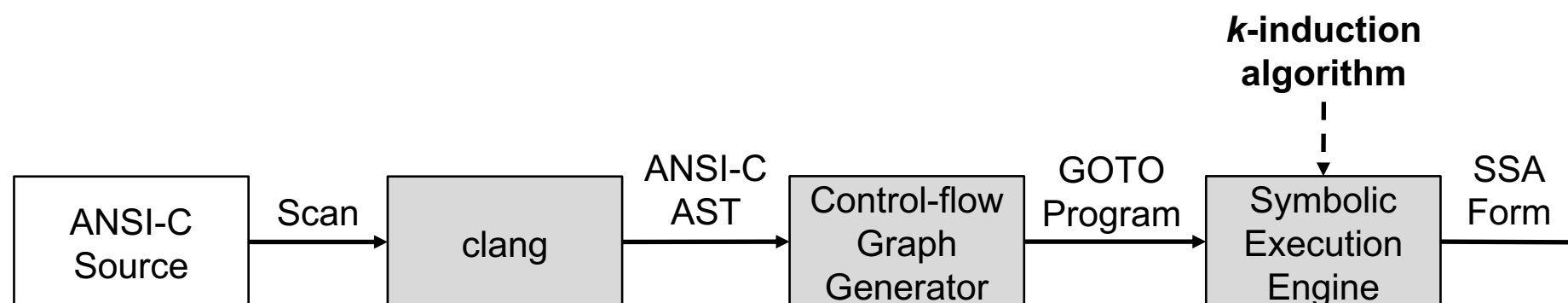
“rectangular” invariant generation based on interval analysis (e.g., $a \leq x \leq b$)



- Abstract-interpretation component from CPROVER
- Only for **integer** variables

ESBMC Architecture

- ESBMC perform a static analysis prior to loop unwinding and (over-)estimate the range that a variable can assume
 - “rectangular” invariant generation based on interval analysis (e.g., $a \leq x \leq b$)



[International Journal on Software Tools for Technology Transfer](#)

February 2017, Volume 19, [Issue 1](#), pp 97–114 | [Cite as](#)

Handling loops in bounded model checking of C programs via *k*-induction

[Authors](#)

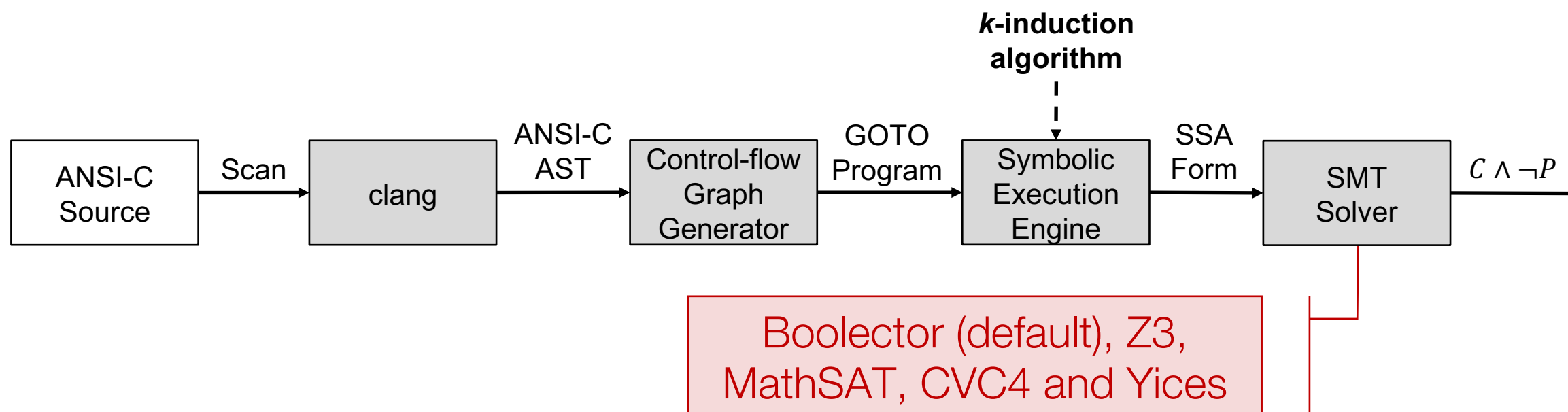
[Authors and affiliations](#)

Mikhail Y. R. Gadelha, Hussama I. Ismail, Lucas C. Cordeiro

ESBMC Architecture

- The back-end is highly configurable and allows the encoding of quantifier-free formulas

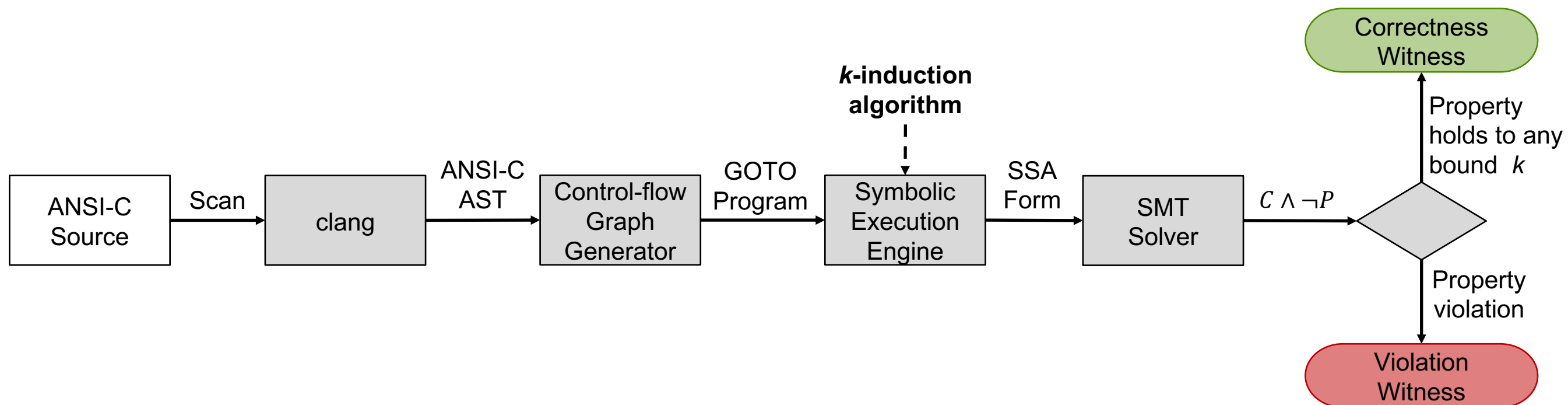
bitvectors, arrays, tuple, fixed-point and floating-point arithmetic (all solvers), and linear integer and real arithmetic (all solvers but Boolector).



ESBMC Architecture

- The back-end is highly configurable and allows the encoding of quantifier-free formulas

bitvectors, arrays, tuple, fixed-point and floating-point arithmetic (all solvers), and linear integer and real arithmetic (all solvers but Boolector).



Strengths & Weaknesses

- Strengths

- Reach-Safety

- ECA (score: 1113)

- Floats (score: 790)

- Heap (score: 300)

- Product Lines (score: 787)



- Falsification (score: 1916)

- Arithmetic, floating point arithmetic

- User-specified assertions

- the use of invariants increases the number of correct proofs in ESBMC by about 7%

- Weaknesses

- need relational analysis that can keep track of relationship between variables

```
unsigned int s = 1;
int main() {
    while (1) {
        unsigned int input = __VERIFIER_nondet_int();
        if (input > 5) {
            return 0;
        } else if (input == 1 && s == 1) {
            s = 2;
        } else if (input == 2 && s == 2) {
            s = 3;
        } else if (input == 3 && s == 3) {
            s = 4;
        } else if (input == 4 && s == 4) {
            s = 5;
        } else if (input == 5 && s > 5) { // unsatisfiable
            __VERIFIER_error(); // property violation
        }
    }
}
```

Simplified safe program extracted from SV-COMP 2018

```
unsigned int s = 1;
int main() {
    while (1) {
        unsigned int input = __VERIFIER_nondet_int();
        if (input > 5) {
            return 0;
        } else if (input == 1 && s == 1) {
            s = 2;
        } else if (input == 2 && s == 2) {
            s = 3;
        } else if (input == 3 && s == 3) {
            s = 4;
        } else if (input == 4 && s == 4) {
            s = 5;
        } else if (input == 5 && s > 5) { // unsatisfiable
            __VERIFIER_error(); // property violation
        }
    }
}
```

**Program under
verification**



**Enable *k*-induction
instead of plain BMC**



**Enable interval
analysis**



`esbmc main.c --k-induction --interval-analysis`

without interval analysis

```
unsigned int s = 1;
int main() {
  while (1) {
    unsigned int
    if (input > 5
      return 0;
    } else if (input < 1)
      s = 2;
    } else if (input < 0)
      s = 3;
    } else if (input < -1)
      s = 4;
    } else if (input < -2)
      s = 5;
    } else if (input == 5 && s > 5) { // unsatisfiable
      __VERIFIER_error(); // property violation
    }
  }
}
```

```
Unwinding loop 1 iteration 49 file svcomp2018.c line 17 function main
Not unwinding loop 1 iteration 50 file svcomp2018.c line 17 function main
Symex completed in: 0.111s (3563 assignments)
Slicing time: 0.008s (removed 2716 assignments)
Generated 1 VCC(s), 1 remaining after simplification (847 assignments)
No solver specified; defaulting to Boolector
Encoding remaining VCC(s) using bit-vector arithmetic
Encoding to solver time: 0.006s
Solving with solver Boolector 3.0.0
Encoding to solver time: 0.006s
Runtime decision procedure: 0.219s
The inductive step is unable to prove the property
Unable to prove or falsify the program, giving up.
VERIFICATION UNKNOWN
```

with interval analysis

1 ASSUME s <= 5 && 1 <= s

```
unsigned int s = 1;
int main() {
  while (1) {
    unsigned int input = __VERIFIER_nondet_int();
    if (input > 5) {
      return 0;
    } else if (input == 1 && s == 1) {
      s = 2;
    } else if (input == 2 && s == 2) {
      s = 3;
    } else if (input == 3 && s == 3) {
      s = 4;
    } else if (input == 4 && s == 4) {
      s = 5;
    } else if (input == 5 && s > 5) { // unsatisfiable
      __VERIFIER_error(); // property violation
    }
  }
}
```

with interval analysis

```
unsigned int s = 1;
int main() {
  while (1) {
    unsigned int input = __VERIFIER_nondet_int();
    if (input > 5) {
      return 0;
    } else if (input == 1 && s == 1) {
      s = 2;
    } else if (input == 2 && s == 2) {
      s = 3;
    } else if (input == 3 && s == 3) {
      s = 4;
    } else if (input == 4 && s == 4) {
      s = 5;
    } else if (input == 5 && s > 5) { // unsatisfiable
      __VERIFIER_error(); // property violation
    }
  }
}
```

1 ASSUME s <= 5 && 1 <= s

2 ASSUME s <= 5 && 1 <= s

with interval analysis

```
unsigned int s = 1;
int main() {
  while (1) {
    unsigned int input = __VERIFIER_nondet_int();
    if (input > 5) {
      return 0;
    } else if (input == 1 && s == 1) {
      s = 2;
    } else if (input == 2 && s == 2) {
      s = 3;
    } else if (input == 3 && s == 3) {
      s = 4;
    } else if (input == 4 && s == 4) {
      s = 5;
    } else if (input == 5 && s > 5) { // unsatisfiable
      __VERIFIER_error(); // property violation
    }
  }
}
```

1 ASSUME s <= 5 && 1 <= s

2 ASSUME s <= 5 && 1 <= s

3 ASSUME s <= 5 && 1 <= s && 6 <= input

with interval analysis

```
unsigned int s = 1;
int main() {
  while (1) {
    unsigned int input = __VERIFIER_nondet_int();
    if (input > 5) {
      return 0;
    } else if (input == 1 && s == 1) {
      s = 2;
    } else if (input == 2 && s == 2) {
      s = 3;
    } else if (input == 3 && s == 3) {
      s = 4;
    } else if (input == 4 && s == 4) {
      s = 5;
    } else if (input == 5 && s > 5) { // unsatisfiable
      __VERIFIER_error(); // property violation
    }
  }
}
```

1 ASSUME $s \leq 5 \ \&\& \ 1 \leq s$

2 ASSUME $s \leq 5 \ \&\& \ 1 \leq s$

3 ASSUME $s \leq 5 \ \&\& \ 1 \leq s \ \&\& \ 6 \leq \text{input}$

4 ASSUME $s == 1 \ \&\& \ \text{input} == 1$

with interval analysis

```

unsigned int s = 1;
int main() {
  while (1) {
    unsigned int input = __VERIFIER_nondet_int();
    if (input > 5) {
      return 0;
    } else if (input == 1 && s == 1) {
      s = 2;
    } else if (input == 2 && s == 2) {
      s = 3;
    } else if (input == 3 && s == 3) {
      s = 4;
    } else if (input == 4 && s == 4) {
      s = 5;
    } else if (input == 5 && s > 5) { // unsatisfiable
      __VERIFIER_error(); // property violation
    }
  }
}

```

Diagram illustrating the execution of the code with interval analysis, showing assumptions at various points:

- 1. ASSUME $s \leq 5 \ \&\& \ 1 \leq s$ (at the start of the `while` loop)
- 2. ASSUME $s \leq 5 \ \&\& \ 1 \leq s$ (at the start of the `if` statement)
- 3. ASSUME $s \leq 5 \ \&\& \ 1 \leq s \ \&\& \ 6 \leq \text{input}$ (at the start of the `return 0;` branch)
- 4. ASSUME $s == 1 \ \&\& \ \text{input} == 1$ (at the start of the `s = 2;` branch)
- 5. ASSUME $s == 2 \ \&\& \ \text{input} == 2$ (at the start of the `s = 3;` branch)

with interval analysis

```

unsigned int s = 1;
int main() {
  while (1) {
    unsigned int input = __VERIFIER_nondet_int();
    if (input > 5) {
      return 0;
    } else if (input == 1 && s == 1) {
      s = 2;
    } else if (input == 2 && s == 2) {
      s = 3;
    } else if (input == 3 && s == 3) {
      s = 4;
    } else if (input == 4 && s == 4) {
      s = 5;
    } else if (input == 5 && s > 5) { // unsatisfiable
      __VERIFIER_error(); // property violation
    }
  }
}

```

1 ASSUME s <= 5 && 1 <= s

2 ASSUME s <= 5 && 1 <= s

3 ASSUME s <= 5 && 1 <= s && 6 <= input

4 ASSUME s == 1 && input == 1

5 ASSUME s == 2 && input == 2

6 ASSUME s == 3 && input == 3

with interval analysis

```

unsigned int s = 1;
int main() {
  while (1) {
    unsigned int input = __VERIFIER_nondet_int();
    if (input > 5) {
      return 0;
    } else if (input == 1 && s == 1) {
      s = 2;
    } else if (input == 2 && s == 2) {
      s = 3;
    } else if (input == 3 && s == 3) {
      s = 4;
    } else if (input == 4 && s == 4) {
      s = 5;
    } else if (input == 5 && s > 5) { // unsatisfiable
      __VERIFIER_error(); // property violation
    }
  }
}

```

Diagram illustrating the execution of the code with interval analysis, showing assumptions at various points:

- 1: ASSUME $s \leq 5 \ \&\& \ 1 \leq s$
- 2: ASSUME $s \leq 5 \ \&\& \ 1 \leq s$
- 3: ASSUME $s \leq 5 \ \&\& \ 1 \leq s \ \&\& \ 6 \leq \text{input}$
- 4: ASSUME $s == 1 \ \&\& \ \text{input} == 1$
- 5: ASSUME $s == 2 \ \&\& \ \text{input} == 2$
- 6: ASSUME $s == 3 \ \&\& \ \text{input} == 3$
- 7: ASSUME $s == 4 \ \&\& \ \text{input} == 4$

with interval analysis

```

unsigned int s = 1;
int main() {
  while (1) {
    unsigned int input = __VERIFIER_nondet_int();
    if (input > 5) {
      return 0;
    } else if (input == 1 && s == 1) {
      s = 2;
    } else if (input == 2 && s == 2) {
      s = 3;
    } else if (input == 3 && s == 3) {
      s = 4;
    } else if (input == 4 && s == 4) {
      s = 5;
    } else if (input == 5 && s > 5) { // unsatisfiable
      __VERIFIER_error(); // property violation
    }
  }
}

```

Diagram illustrating the execution of the code with interval analysis, showing assumptions at various points:

- 1: ASSUME $s \leq 5 \ \&\& \ 1 \leq s$
- 2: ASSUME $s \leq 5 \ \&\& \ 1 \leq s$
- 3: ASSUME $s \leq 5 \ \&\& \ 1 \leq s \ \&\& \ 6 \leq \text{input}$
- 4: ASSUME $s == 1 \ \&\& \ \text{input} == 1$
- 5: ASSUME $s == 2 \ \&\& \ \text{input} == 2$
- 6: ASSUME $s == 3 \ \&\& \ \text{input} == 3$
- 7: ASSUME $s == 4 \ \&\& \ \text{input} == 4$
- 8: ASSUME $s \leq 5 \ \&\& \ 1 \leq s \ \&\& \ \text{input} \leq 5$

with interval analysis

```

unsigned int s = 1;
int main() {
  while (1) {
    unsigned int input = __VERIFIER_nondet_int();
    if (input > 5) {
      return;
    } else if (s == 1) {
      s = 2;
    } else if (s == 2) {
      s = 3;
    } else if (s == 3) {
      s = 4;
    } else if (s == 4) {
      s = 5;
    } else if (s == 5) {
      __VERIFIER_assert(input <= 5);
    }
  }
}

```

1 ASSUME s <= 5 && 1 <= s

2 ASSUME s <= 5 && 1 <= s

3 ASSUME s <= 5 && 1 <= s && 6 <= input

```

*** Checking inductive step
Starting Bounded Model Checking
Unwinding loop 1 iteration 1 file svcomp2018.c line 17 function main
Not unwinding loop 1 iteration 2 file svcomp2018.c line 17 function main
Symex completed in: 0.001s (82 assignments)
Slicing time: 0.000s (removed 22 assignments)
Generated 1 VCC(s), 1 remaining after simplification (60 assignments)
No solver specified; defaulting to Boolector
Encoding remaining VCC(s) using bit-vector arithmetic
Encoding to solver time: 0.000s
Solving with solver Boolector 3.0.0
Encoding to solver time: 0.000s
Runtime decision procedure: 0.001s
BMC program time: 0.003s

```

VERIFICATION SUCCESSFUL

Solution found by the inductive step (k = 2)

input == 1

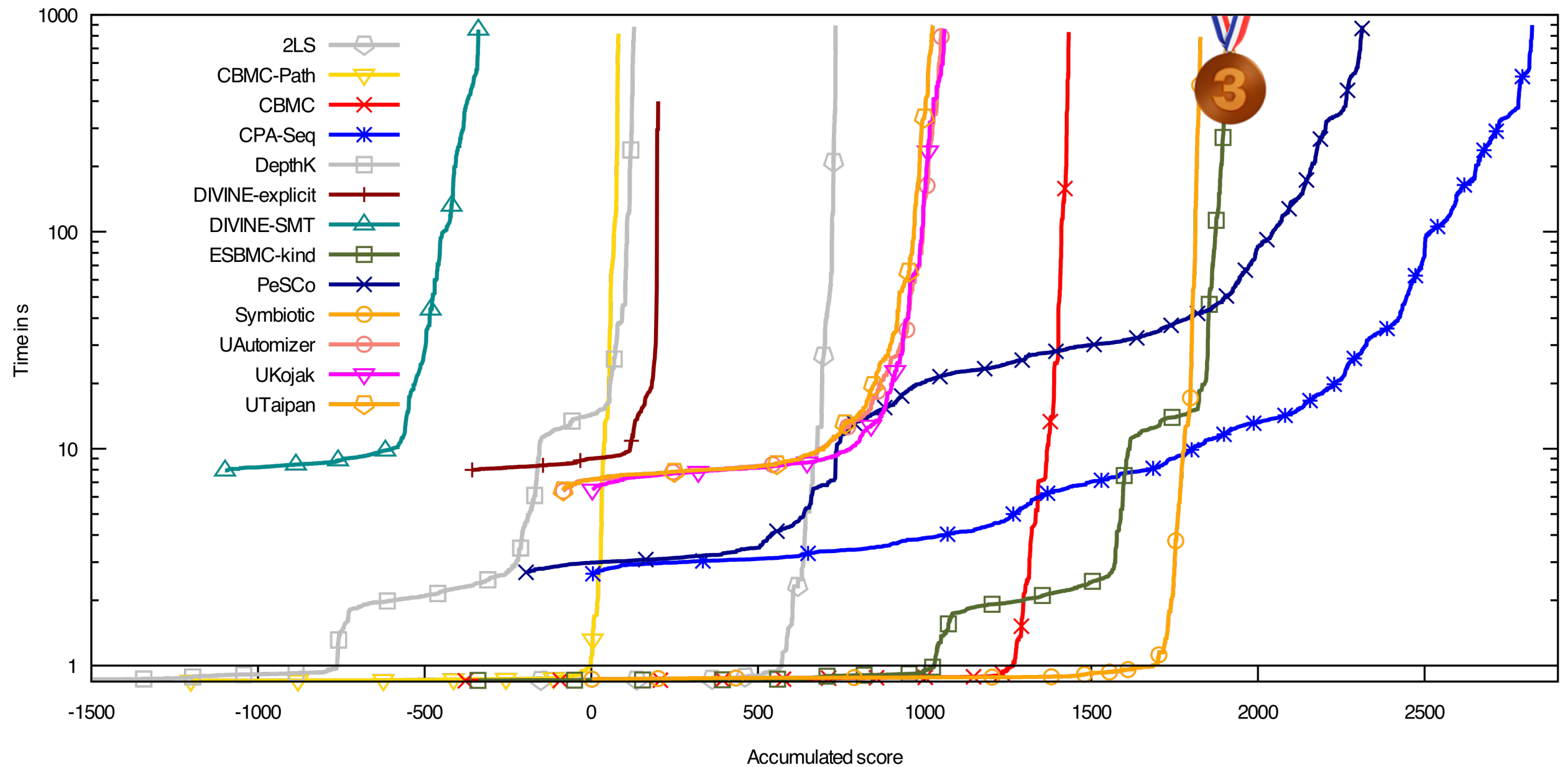
input == 2

input == 3

input == 4

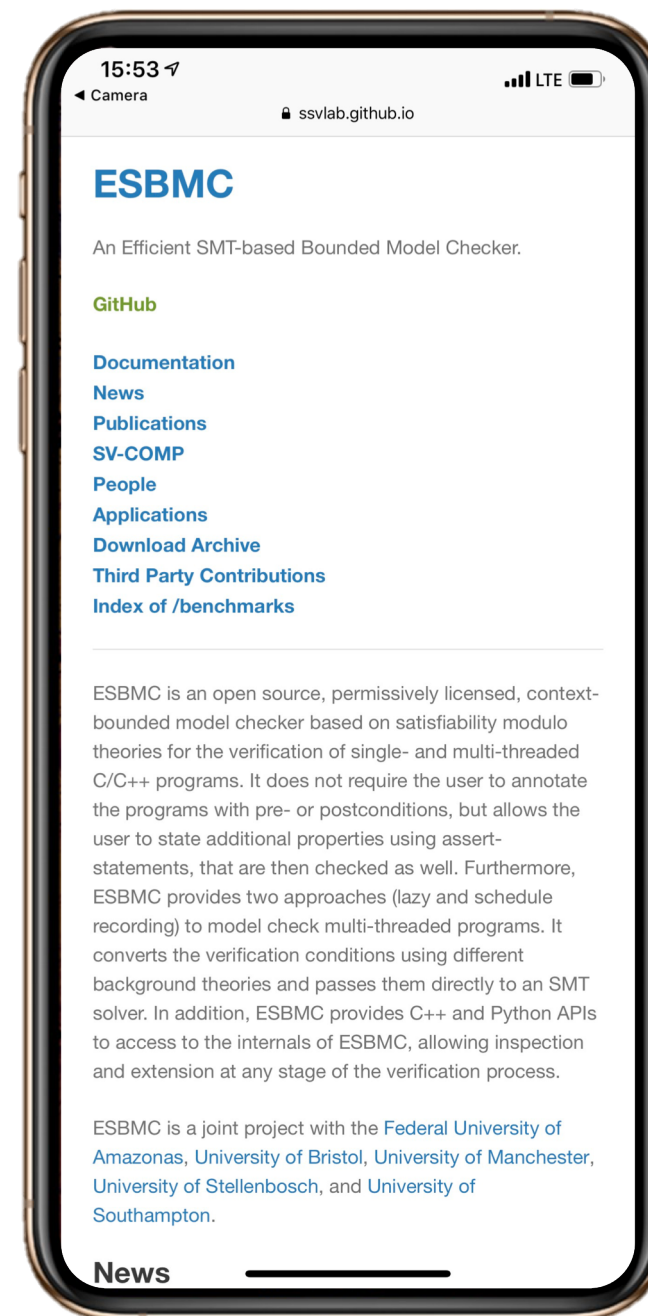
input <= 5

Falsification



Thank you!

More information available at <http://esbmc.org/>



JBMC: A Bounded Model Checking Tool for Verifying Java Bytecode

Lucas Cordeiro
Pascal Kesseli
Daniel Kroening
Peter Schrammel
Marek Trtik



Computer Aided Verification 2018

Why?

Java and JVM languages:

- Most widely used
- Established software development culture

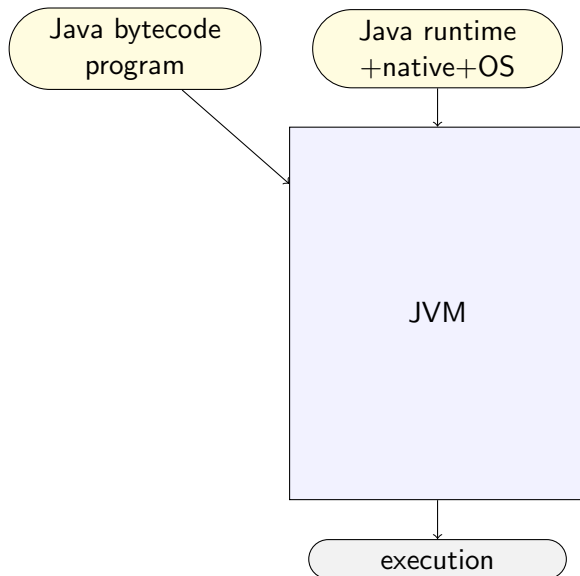
Only few model checking tools available:

- Symbolic JPF (Anand et al, TACAS'07)
- JayHorn (Kahsai et al, CAV'16)

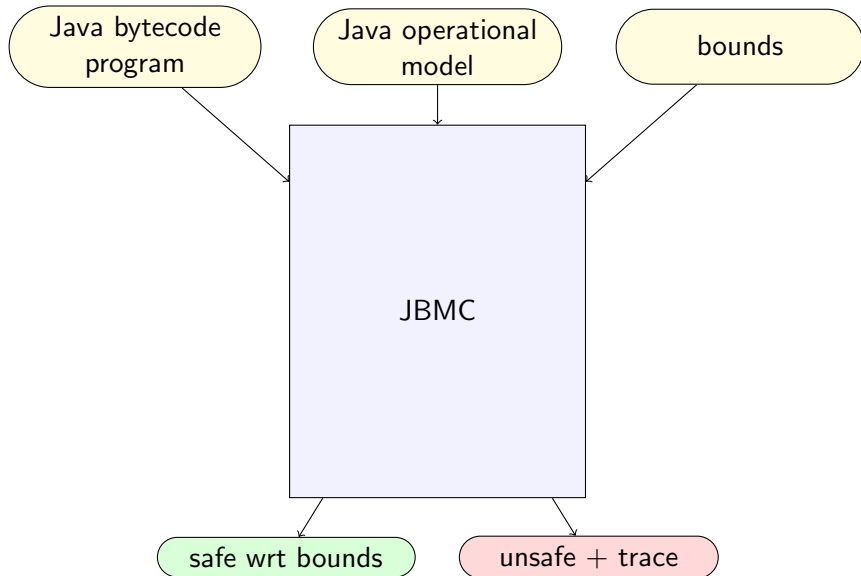
Many applications of BMC:

- Bug finding
- Program synthesis
- Test generation
- ...

JVM vs JBMC

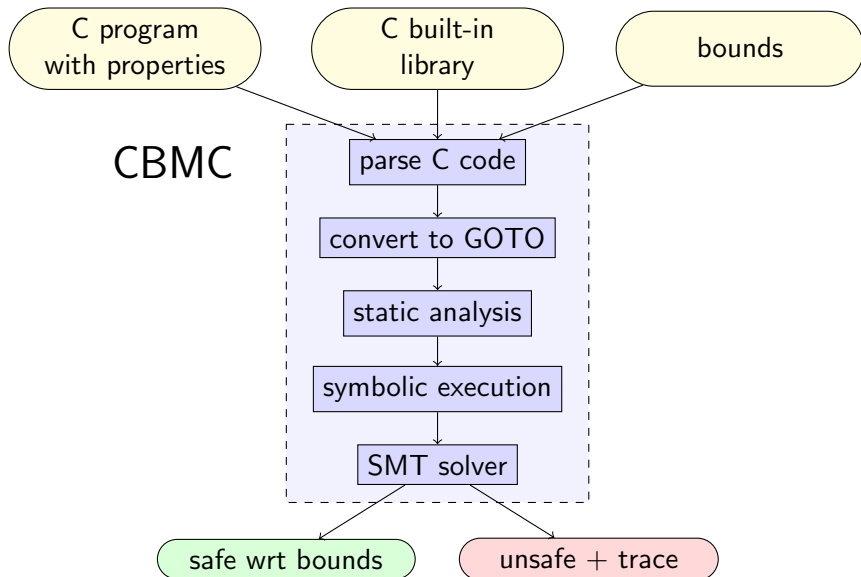


JVM vs JBMC



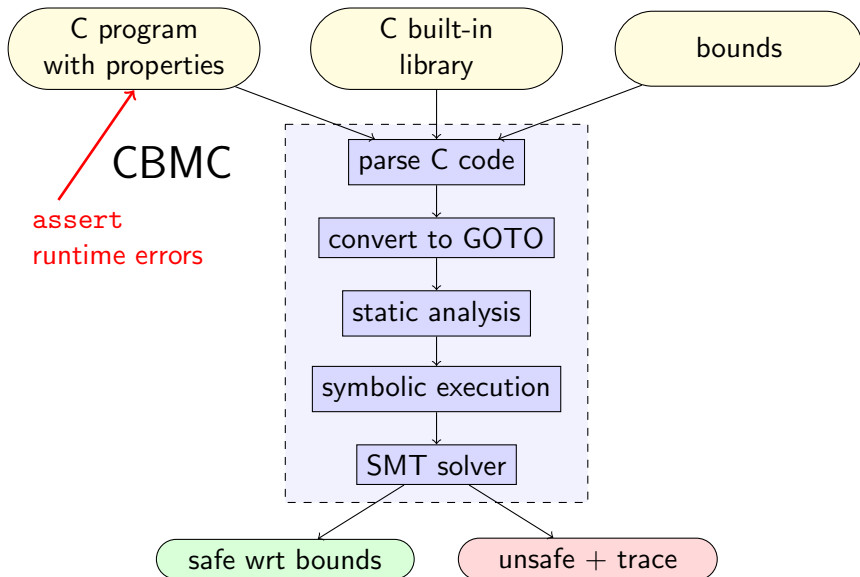
CBMC

Clarke, Kroening & Lerda, TACAS'04



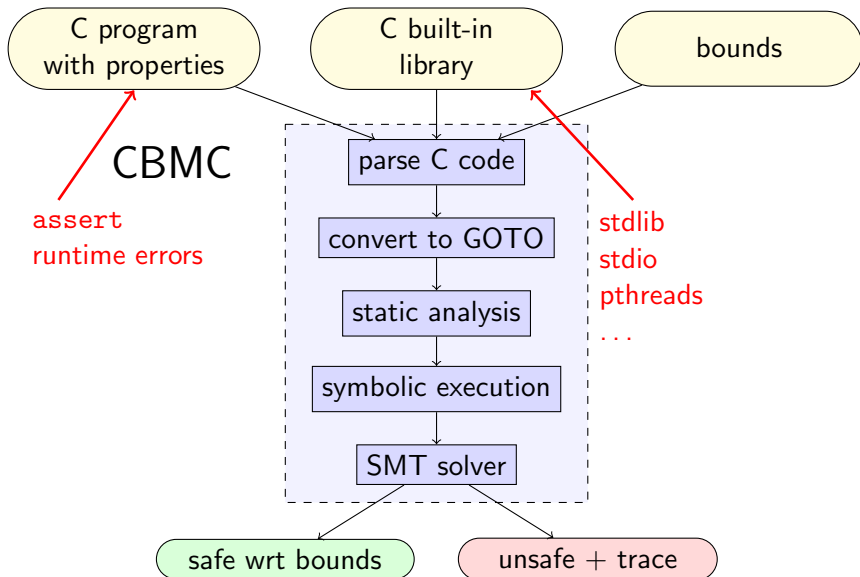
CBMC

Clarke, Kroening & Lerda, TACAS'04



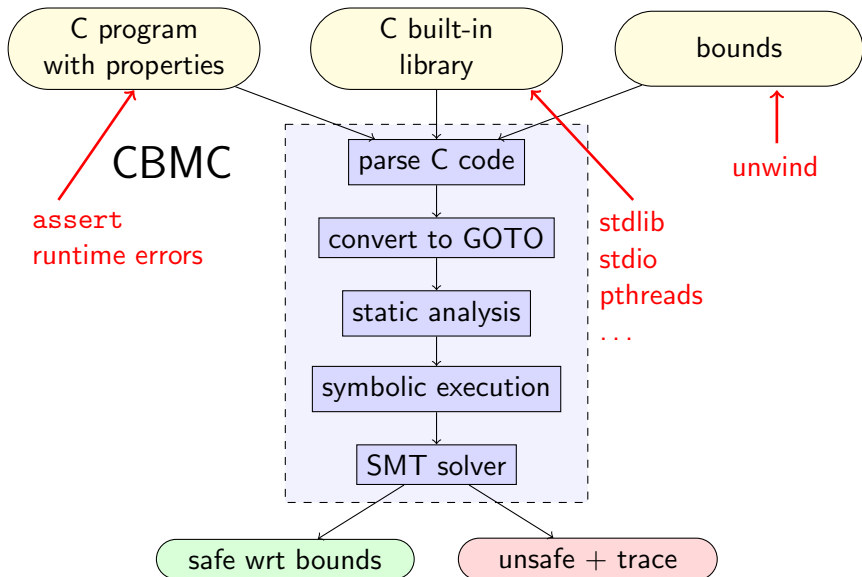
CBMC

Clarke, Kroening & Lerda, TACAS'04



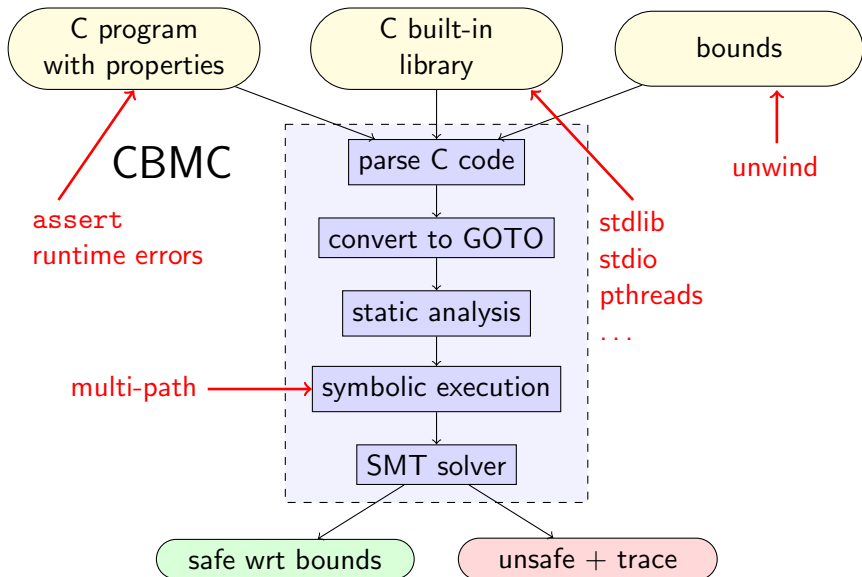
CBMC

Clarke, Kroening & Lerda, TACAS'04



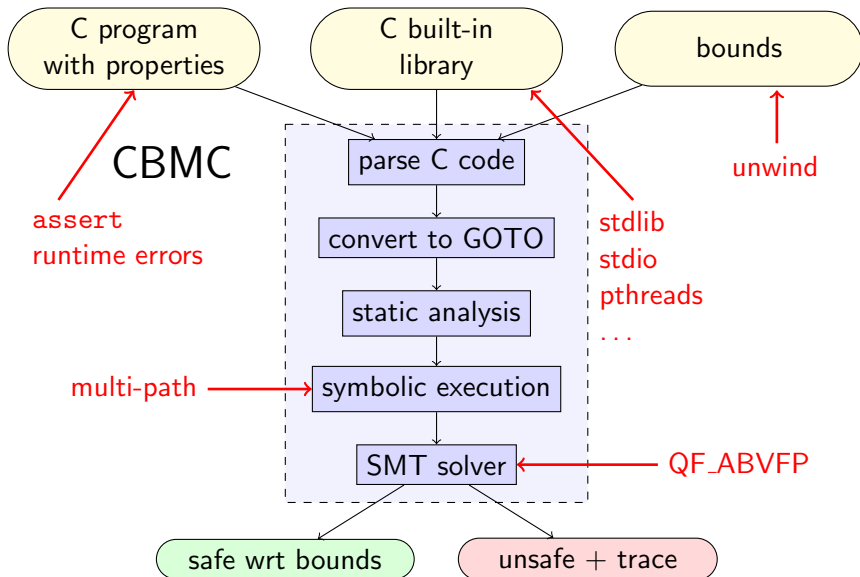
CBMC

Clarke, Kroening & Lerda, TACAS'04

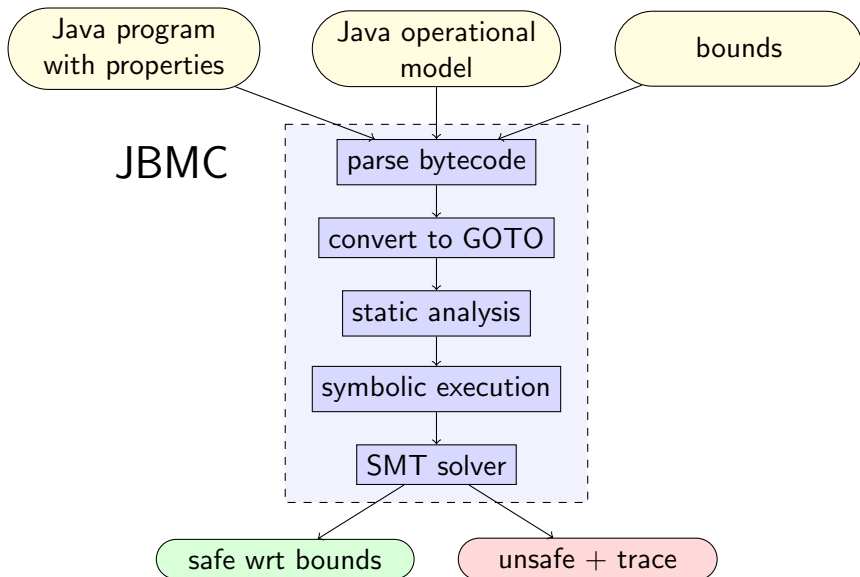


CBMC

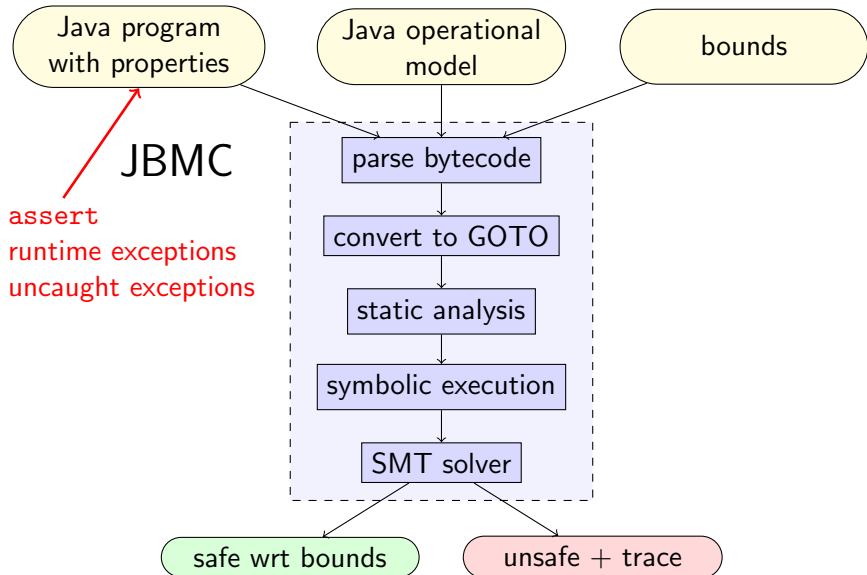
Clarke, Kroening & Lerda, TACAS'04



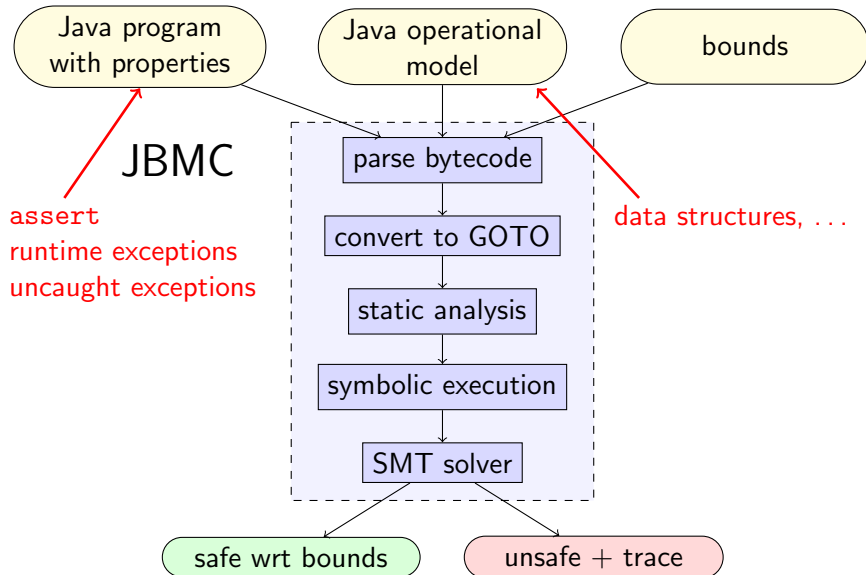
JBMC



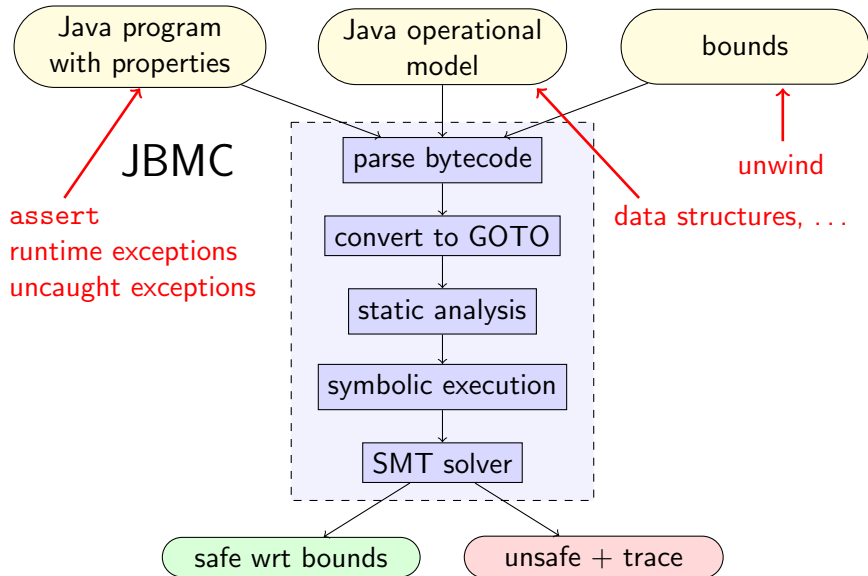
JBMC



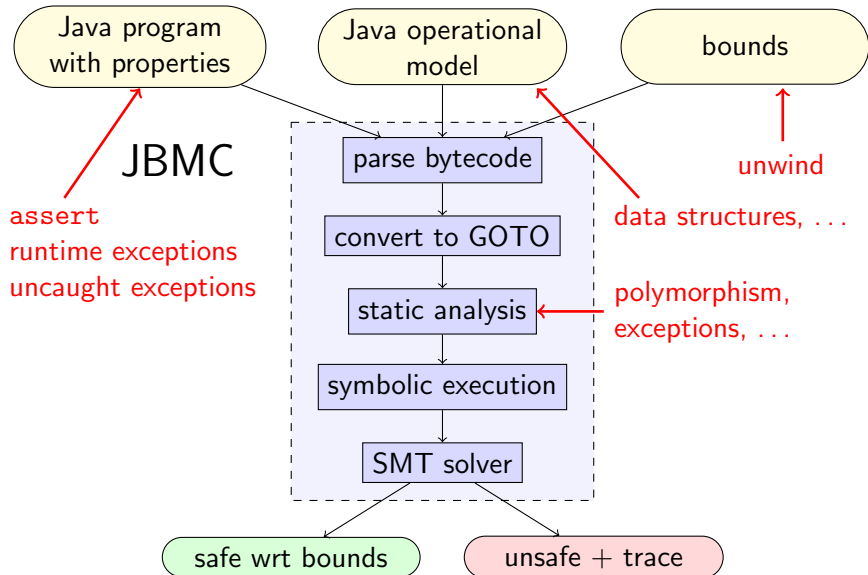
JBMC



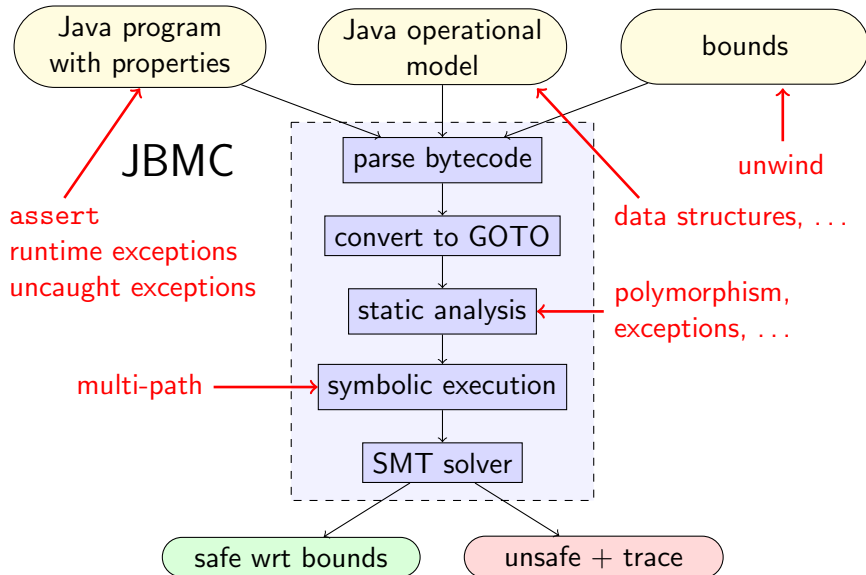
JBMC



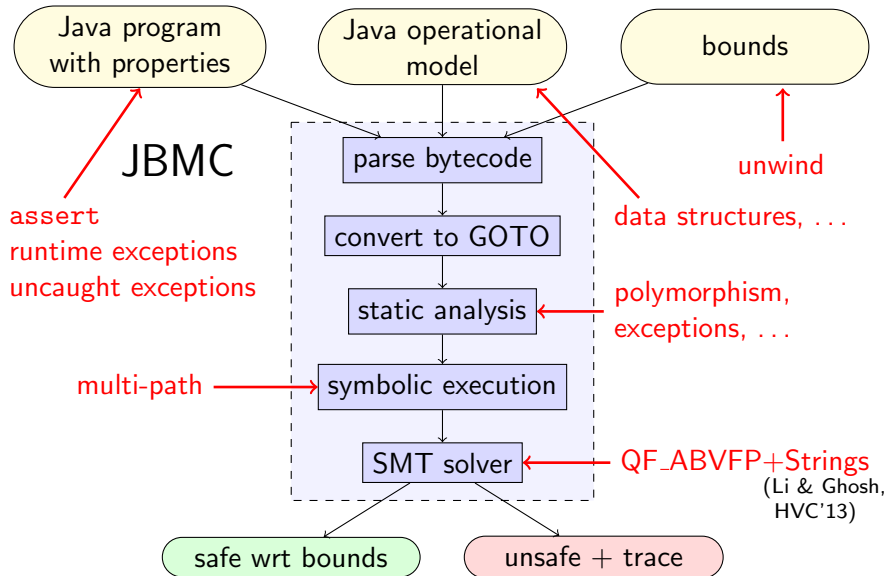
JBMC



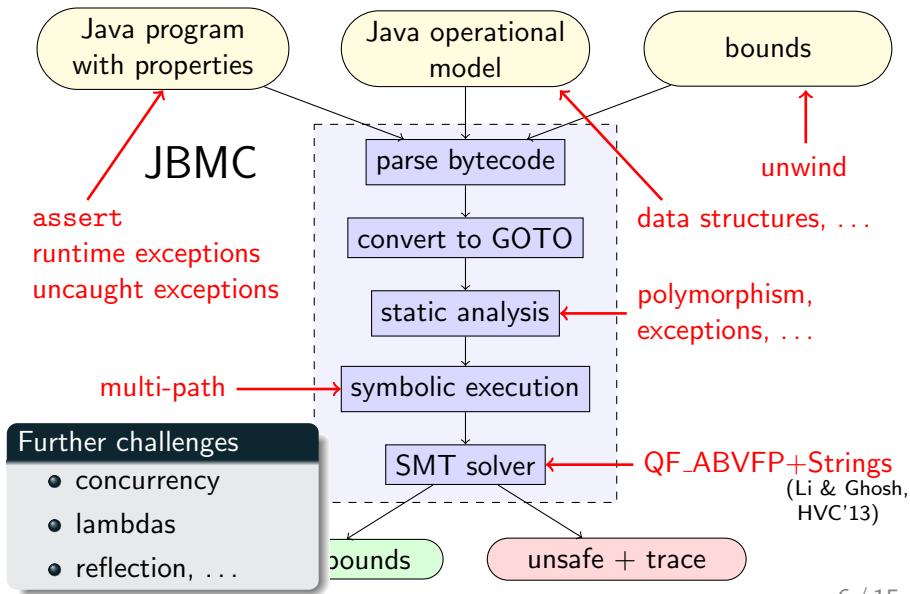
JBMC



JBMC



JBMC



JBMC in Action

```
public class Calculator {
    public static void main(String[] args) {
        if(args.length != 3 || args[1].length() != 1) {
            System.err.println("invalid input");
            return;
        }

        int a = Integer.parseInt(args[0]);
        char op = args[1].charAt(0);
        int b = Integer.parseInt(args[2]);

        int result = 0;
        switch(op) {
            case '+': result = a + b; break;
            case '-': result = a - b; break;
            case '*': result = a * b; break;
            case '/': result = a / b; break;
        }

        System.out.println("Result: "+result);
    }
}
```

JBMC in Action

```
$ jbmcc Calculator.class --classpath core-models.jar:. --trace  
--throw-runtime-exceptions
```

```
...
```

```
Runtime decision procedure: 0.980s
```

```
...
```

```
dynamic_object4={ '-', '0', '4' } // args[0].data
```

```
...
```

```
dynamic_object6={ '/' } // args[1].data
```

```
...
```

```
dynamic_object8={ '0' } // args[2].data
```

```
...
```

```
Calculator.java line 17 function Calculator.main
```

```
dynamic_object36={... .@class_identifier="java.lang.ArithmeticException" ...}
```

```
...
```

```
uncaught_exception=&dynamic_object36
```

Violated property:

file Calculator.java line 3 function Calculator.main

no uncaught exception

uncaught_exception == null

JBMC in Action

```
public class MyTranslator {
    static abstract class Translator {
        abstract String translate(String text);
        static Translator build(String language) {
            if("Chinese".equals(language))
                return new ChineseTranslator();
            return null;
        }
    }
    static class ChineseTranslator extends Translator {
        String translate(String text) {
            if(text.toLowerCase().contains("welcome to oxford"))
                return "欢迎来到牛津";
            return "I don't understand";
        }
    }
    public static void main(String[] args) {
        if(args.length < 2)
            return;
        Translator translator = Translator.build(args[0]);
        if(translator == null)
            return;
        String translatedText = translator.translate(args[1].trim());
        assert(!"欢迎来到牛津".equals(translatedText));
    }
}
```

JBMC in Action

```
$ jbmc MyTranslator.class --classpath core-models.jar:. --trace  
  --max-nondet-string-length 50
```

```
...
```

```
Runtime decision procedure: 0.786s
```

```
...
```

```
dynamic_object4={ 'C', 'h', 'i', 'n', 'e', 's', 'e' } // args[0].data
```

```
...
```

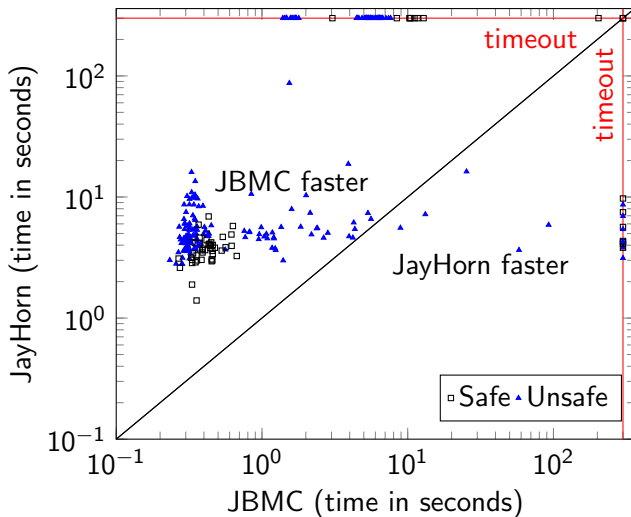
```
dynamic_object6={ '\u0010', '\u0017', 'w', 'e',  
                  'w', 'E', 'L', 'C', 'O', 'M', 'E', ' ',  
                  'T', 'o', ' ',  
                  'o', 'x', 'f', 'o', 'r', 'D',  
                  'x', 'x', 'x', 'x', 'x', 'x', 'x', 'x', 'x', 'x',  
                  'x', ' ', ' ', ' ' } // args[1].data
```

```
...
```

Violated property:

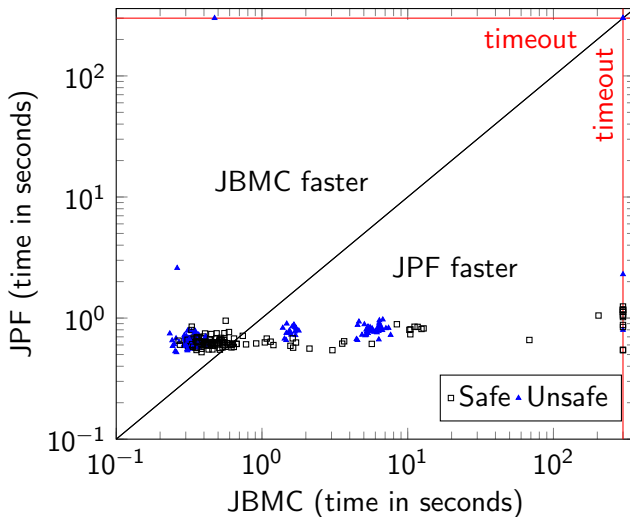
assertion at file MyTranslator.java line 31 function MyTranslator.main

Comparison with JayHorn 0.5.1



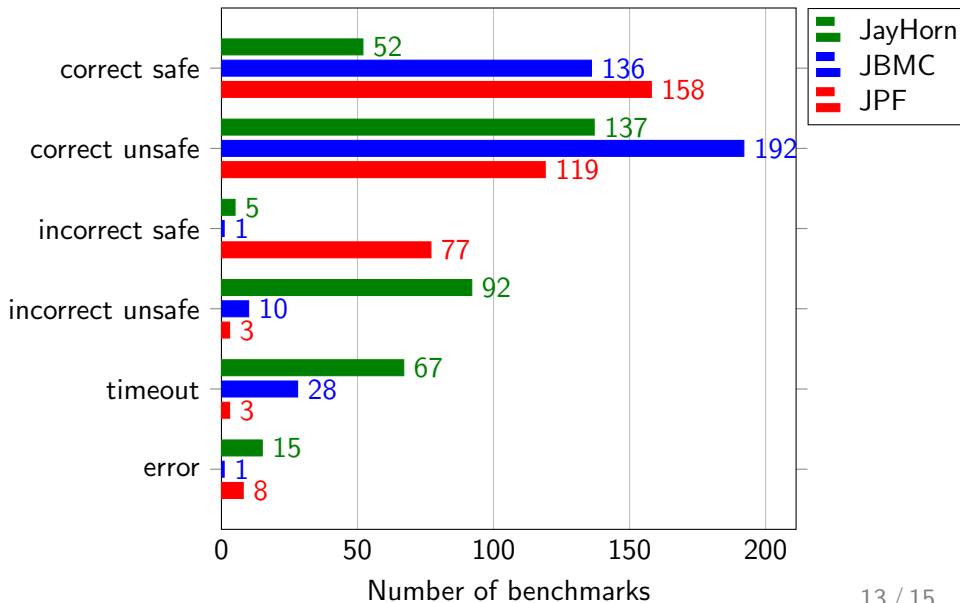
timeout 300s, 15GB

Comparison with JPF v32

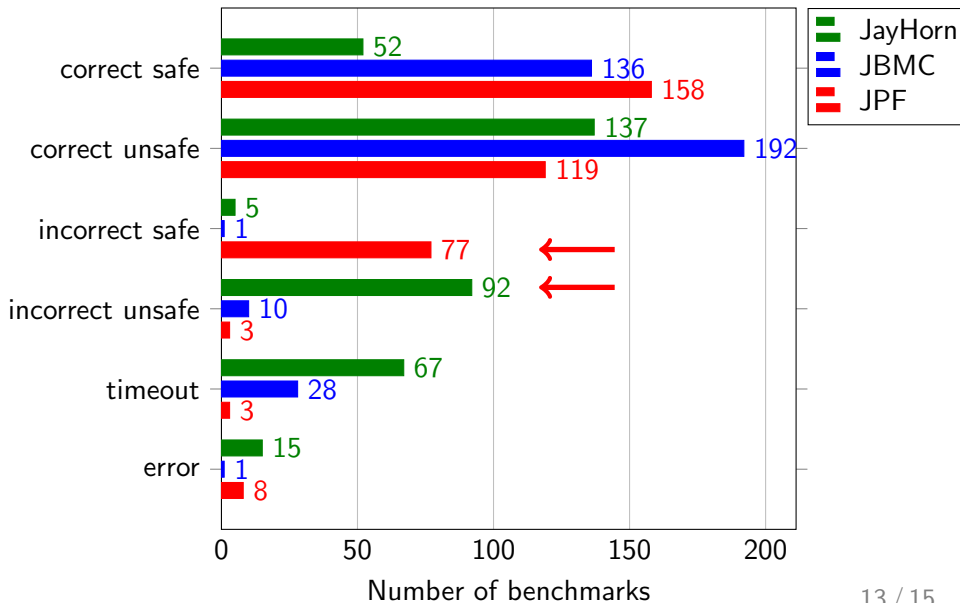


timeout 300s, 15GB

Comparison



Comparison



Java @ TACAS SV-COMP 2019

Objectives:

- More languages in SV-COMP
 - Standard benchmark set
 - Comparability
 - Reproducibility
 - Re-use existing benchmarking infrastructure
- 3 tools already integrated: JPF, Jayhorn, JBMC

Timeline:

- September 2018: Contribution of benchmarks
- October 2018: Tool submission

Watch out on sv-comp.sosy-lab.org

Subscribe to sv-comp@googlegroups.com

Tool	JPF 32				Jayhorn 32				JBMC 32			
	memory	time	memory	time	memory	time	memory	time	memory	time	memory	time
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32				JBMC 32			
Tool	JBMC 32				JBMC 32							

Download JBMC and contribute!

www.cprover.org/jbmc

Java @ TACAS SV-COMP 2019

- Contribute and participate!
- sv-comp.sosy-lab.org

www.diffblue.com

- Jobs in program analysis, verification and machine learning!

